



Role-Oriented Graph Auto-encoder Guided by Structural Information

Xuan Guo¹, Wang Zhang¹, Wenjun Wang¹, Yang Yu¹, Yinghui Wang¹,
and Pengfei Jiao^{1,2}(✉) 

¹ College of Intelligence and Computing, Tianjin University, Tianjin 300350, China
{guoxuan, wangzhang, wjwang, yuyangyy, wangyinghui}@tju.edu.cn

² Center of Biosafety Research and Strategy, Tianjin University,
Tianjin 300092, China
pjiao@tju.edu.cn

Abstract. Roles in a complex network usually represent the local connectivity patterns of nodes, which reflect the functions or behaviors of corresponding entities. Role discovery has great meaning for understanding the formation and evolution of networks. While the importance of role discovery in networks has been realized gradually, a variety of approaches of role-oriented network representation learning are proposed. Almost all the existing approaches are dependent on manual high-order structural properties which are always fragmentary. They suffer from unstable performances and poor generalization ability, because their hand-craft structural features sometimes miss the characteristics of different networks. In addition, graph neural networks (GNNs) have great potential to automatically capture structural properties, but it is hard to be given the rein to for the difficulty of designing role-oriented unsupervised loss. To overcome these challenges, we provide an idea that leverage low-dimensional extracted structural features as guidance to train graph neural networks. Based on the idea, we proposed GAS, a novel graph auto-encoder guided by structural information, to learn role-oriented representations for nodes. Results of extensive experiments show that GAS has better performance than other state-of-the-art approaches.

Keywords: Role discovery · Network embedding · Graph convolution networks · Graph auto-encoder

1 Introduction

Graph or Network is the natural representation structure of irregular data in numerous application domains, including social networks [28], proteomics [31], etc. Specifically, nodes and edges in a network are used to represent real-world entities and their relationships. In this way, different problems in many domains can be transformed into corresponding research problems on networks. While

X. Guo and W. Zhang—These authors contributed equally to this work.

© Springer Nature Switzerland AG 2020

Y. Nah et al. (Eds.): DASFAA 2020, LNCS 12113, pp. 466–481, 2020.

https://doi.org/10.1007/978-3-030-59416-9_28

modeling complex systems in reality, networks contain a lot of useful hidden information which is worth mining out and analyzing considerably. **Community Detection** [11] and **Role Discovery** [21, 24] are such two research fields of mining network information at the mesoscopic level.

Community detection and role discovery are two kinds of graph clustering problems based on different criteria. As shown in Fig. 1, the nodes play different roles as reflected in their local connectivity patterns while forming communities having more connections inside than outside. Correspondingly, real-world entities always perform various functions and behaviors in groups, such as employers and employees at different positions forming companies. Naturally, studying communities can help to learn the shared interests and objectives of entities while studying roles are conducive to capturing the distinctions between relationships. Therefore, communities and roles are of great significance for understanding the formation and evolution of networks. However, community detection has been profoundly studied for a long time while role discovery has attracted little but increasing attention in recent years.

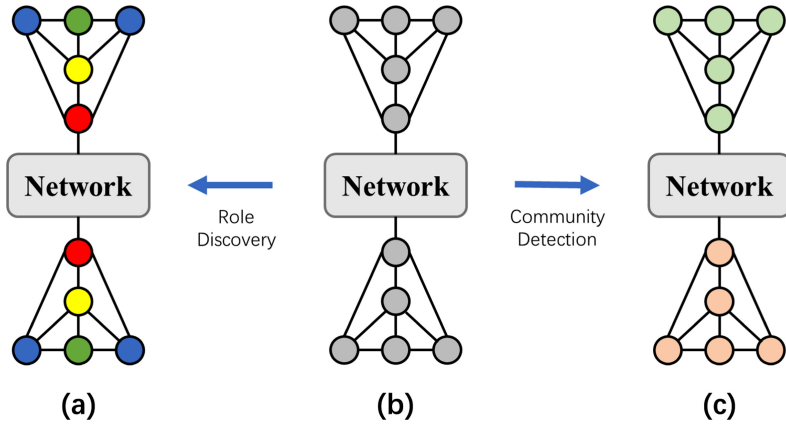


Fig. 1. Comparison between role discovery and community detection. (a) The result of role discovery. (b) The original network. (c) The result of community detection.

Essentially, role discovery is the process of capturing structural properties of nodes. Thus, classic metrics computed based on node-centric structure, such as PageRank [18] and other node centralities, could be considered as role measures. But these metrics can only represent roles from a particular viewpoint and usually have high computational complexity, which limit their applications for complex and massive analysis and machine learning tasks on large-scale networks.

Fortunately, **Network Embedding** (also known as **Network Representation Learning**), which uses low-dimensional vectors to preserve original node information in networks, has shown great power on representing large-scale networks. Compared with the above metrics, the learned embeddings are more

versatile representations. While losing complex graph structure and preserving much information in vectors, the embeddings can be easily and effectively utilized for large amounts of tasks of network analysis and machine learning. However, most of existing network embedding algorithms [5, 19, 25] are based on proximity of nodes, which means the closer nodes in the network have more similar embeddings. In other words, these algorithms are oriented to communities. As the importance of roles has been realized gradually, some role-oriented network embedding approaches have been proposed recently.

The aim of role-oriented network embedding is to encode node-centric structural information and transform the structural similarities between nodes into the geometric relationships in embedding space. Guided by the purpose, almost all of role-oriented network embedding approaches are in two steps: **(S1)** Capture the structural properties of nodes. **(S2)** Mapping structural properties and similarities into embeddings. Most role-oriented embedding approaches leverage high-order structural features to capture structural information. For example, role2vec [1] and HONE [23] leverage small induced subgraphs called motifs while RolX [8], GLRD [3] and DMER [12] take advantages of a recursive feature aggregation algorithm ReFeX [9]. DRNE [27] uses layer normalized LSTMs to represent nodes on sorted neighbors. Meanwhile, random walks [1, 20], matrix factorization [7, 8] and graph neural networks [12, 27] are commonly used as mapping methods.

Evidently, obtaining high-quality structural properties is the key to learning role-oriented embeddings. However, the methods of extracting structural properties that most role-oriented embedding approaches use have a number of limitations. First, those extraction methods are manual, with the result that obtained information is quite fragmentary. While the characteristics decisively determining roles vary in different networks, the extracted features are not generally applicable. Second, some unsupervised learning methods would make generated embeddings overfitting the input features while ignoring the graph structure. Though hand-craft structural features are intuitive to understand and easy to handle, they suffer from massive information loss. Third, extraction of high-order structural features such as motifs could be quite time-consuming.

What's more, **graph neural networks (GNNs)** have the natural ability to learn structural properties due to its propagation mechanism on the edges. However, it is pretty difficult to design the role-oriented unsupervised loss. Though some works tried in different ways including reconstructing input features and approximating embeddings by recursively aggregation on neighborhood, they still face limitations mentioned above for over-dependency on manual extraction process. What role-oriented GNN models need is only little guidance with which they can extract high-quality structural information automatically.

To address aforementioned problems, we propose **GAS**, a novel graph auto-encoder guided by structural information. In GAS, we use graph convolutional layers [14] to capture structural properties. We replace the symmetric normalized adjacency matrix in each graph convolutional layer with the unnormalized adjacency matrix to distinguishing local structures more powerfully. We leverage

structural features as guidance information instead of inputs so that our method can greatly alleviate the problems caused by over-dependence on hand-craft features. Additionally, the features we use are obtained by aggregating neighbor’s primary features only once and their dimensions are very low. Without lots of unnecessary computations, our model is highly efficient.

In summary, the contributions of our paper are as follows:

1. We are the first to provide the idea of using structural features as guidance information to train role-oriented graph neural networks.
2. We propose GAS, a novel graph auto-encoder guided by structural information for role-oriented tasks. We leverage features in very low dimensions to GAS for high efficiency and effectiveness.
3. While performing better than other state-of-the-art embedding methods in a variety of experiments on several real-world datasets, the correctness of our guidance idea and the effectiveness of our approach are verified.

2 Related Work

Graph is irregular structure and real-world networks are always in large-scale and sparse. Therefore, it is too difficult to utilize graph structure data directly as inputs of complex and massive network tasks. Inspired by word embedding methods, which generate dense representations in low dimensions for sparse distributed words, **Network Embedding** methods are proposed to encode nodes into low-dimensional embedding space.

DeepWalk [19] is the first to introduce the classic language model Skip-Gram [17] to network representation learning. It leverages random walks to generate sequences composed of nodes as the inputs of Skip-Gram. Then Skip-Gram produces representations of nodes. On the basis of DeepWalk, Node2vec [5] makes random walks biased by adding two hyper-parameters in order to capture both homogeneity and structural properties of nodes. However, embeddings of the nodes that are close in the network are still similar due to the similar node contexts. To be applicable for large-scale networks, LINE [25] uses an objective function that preserves direct links (first-order proximity) and shared neighbors (second-order proximity) of nodes and an edge-sampling algorithm for optimizing the objective. Therefore, these methods are all designed for capturing proximity of nodes and not feasible for role-oriented tasks.

Some role-oriented network embedding approaches have been proposed recently. ReFeX [9] extracts local and egonet features and aggregates the features of neighbors recursively. As an efficient method of high-order structural feature extraction, ReFeX is applied extensively in many other role-oriented embedding approaches. For example, RolX [8] leverages ReFeX to extract structural features and generates embeddings via Non-negative Matrix Factorization. Soon afterwards, GLRD [3] extends RolX by adding sparsity and diversity constraints to the NMF formulation. Similarly, xNetMF in REGAL [7] firstly obtains structural features by counting of node degrees of k-hop neighbors of each node. Then

it uses Singular Value Decomposition to encode the similarity calculated based on both structure and attributes to representations.

There are several approaches based on random walks. Role2vec [1] designs a feature based random walk for learning role-oriented embeddings. It replaces the sequences of nodes in DeepWalk with sequences of motif-based feature values so that structural information could be kept in the representations. In contrast to Role2vec which leveraging features to random walks in a direct way, Struc2vec [20] constructs a hierarchy of complete graphs by transforming degree-based structural similarities to weights of edges. After construction, Skip-Gram are trained on the multi-layer network.

Graph neural networks have great potential to capture local connectivity patterns due to their propagation mechanisms on edges. Nevertheless, to our best known, only two role-oriented embedding methods leverage graph neural networks. DRNE [27] uses layer normalized LSTMs [10] to handle graph data entered. In essence, it is an semi-manual method of capturing structural properties. DRNE also defines a recursive aggregation process to learn regular equivalence of nodes. DMER [12], a deep mutual encode model combining graph convolutional networks [14] and feature-based auto-encoder, is an attempt to reduce dependency on manual processes. It should be noted that the propagation mechanisms could be a double-edged sword. A large number of aggregation processes might smooth embeddings of connected nodes to much, which is really harmful to role-oriented embeddings.

Additionally, HONE [23] and GraphWave [2] use different diffusion methods to learn the local connectivity patterns of nodes.

It's apparent almost all of above role-oriented embedding approaches can be summed up in two steps: structural properties extraction and mapping. While the quality of extracted properties determines the effectiveness of learned representations, these approaches face lots of problems caused by their mutual extraction methods.

3 Method

In this section, we introduce the notations and the details of our proposed role-oriented network embedding framework named GAS. As the overview is shown in Fig. 2, our framework are roughly divided in three parts: **(a)** Extract structural features as the guidance information; **(b)** Use a graph auto-encoder for encoding nodes to role-oriented embeddings which are reconstructed to features by the decoder; **(c)** Compute the loss based on guidance features and reconstructed features for training the graph auto-encoder.

3.1 Notations

Given a undirected unweighted network $G = (V, E)$, where $V = \{v_1, \dots, v_n\}$ is the set of n nodes and $E \subseteq V \times V$ is the set of edges among the nodes. For each node $v \in V$, the set of its neighborhood is denoted as $\mathcal{N}(v) = \{u | (v, u) \in E\}$.

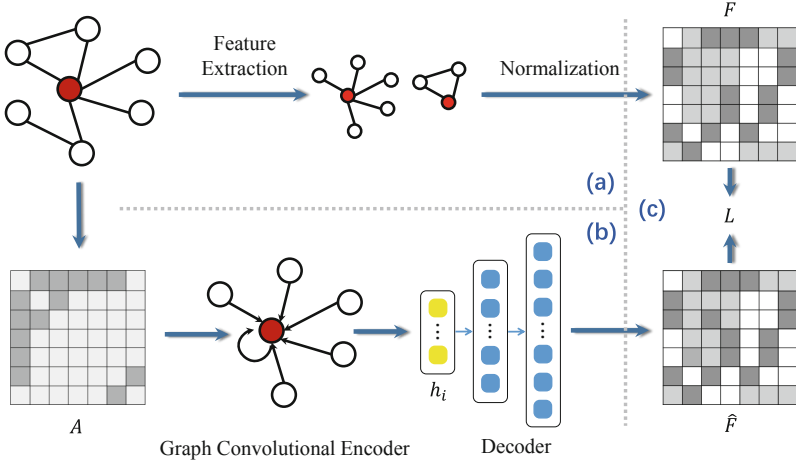


Fig. 2. Overview of our proposed GAS framework. (a): Structural feature extraction. (b): Encoder-decoder architecture. (c): Loss computation based on artificially extracted features and features reconstructed by decoder.

Table 1. Main notations and their definitions.

Notation	Definition
$G = (V, E)$	The graph G with node set V and edge set E
$A \in \mathbb{R}^{n \times n}$	The adjacency matrix of G
$D \in \mathbb{R}^{n \times n}$	The degree matrix of G
$F \in \mathbb{R}^{n \times d}$	The extracted structural matrix of G
$\mathcal{N}(v) = \{u (v, u) \in E\}$	The set of node v 's neighbors G
$\mathcal{E}(v) = (V_{\mathcal{E}(v)}, E_{\mathcal{E}(v)})$	The egonet of node v
$T(v)$	The set of triangles node v participates in

$A \in \mathbb{R}^{n \times n}$ is the adjacency matrix of G . $A_{ij} = 1$ if v_i and v_j are linked in G , otherwise $A_{ij} = 0$. $\mathcal{E}(v) = (V_{\mathcal{E}(v)}, E_{\mathcal{E}(v)})$ denotes the egonet of node v , where $V_{\mathcal{E}(v)} = \mathcal{N}(v) \cup \{v\}$ and $E_{\mathcal{E}(v)} = \{(u, u') | (u, u') \in E \wedge u, u' \in V_{\mathcal{E}(v)}\}$ are the sets of nodes and edges in $\mathcal{E}(v)$ respectively. $T(v) = \{\{v, u, u'\} | (v, u), (v, u'), (u, u') \in E\}$ means the set of triangles node v participates in. The symbol of degree matrix is $D_{ii} = \sum_j A_{ij}$. $F \in \mathbb{R}^{n \times m}$ represents the extracted structural feature matrix, where each row F_i is the m -dimensional feature vector of node v_i . The notations mainly used in this paper are summarized in Table 1.

3.2 Feature Extraction

Role-oriented network representation learning is unsupervised. While treating hand-craft structural features as inputs, the generated embeddings would fit features. This kind of dependence on features is too strong and makes embedding

approaches not generally applicable. To alleviate the problems, we choose to use extracted features as guidance information for training our role-oriented graph auto-encoder.

We draw on the experience of ReFeX [9] which aggregates the simple local and egonet features of neighbors recursively. For each node v , extracted local and egonet features are enumerated as follows:

- (F1) The degree of v : $f_1 = |\mathcal{N}(v)|$.
- (F2) The number of edges in the egonet of v : $f_2 = |E_{\mathcal{E}(v)}|$.
- (F3) The sum of node’s degree in the egonet of v : $f_3 = \sum_{\mathcal{E}(v)} |\mathcal{N}(u)|$.
- (F4) The approximate proportion of within-egonet edges to all the edges entering and leaving the egonet of v : $f_4 = f_2/f_3$.
- (F5) The approximate proportion of non-egonet edges to all the edges entering and leaving the egonet of v : $f_5 = 1 - f_2/f_3$.
- (F6) The number of triangles v participates in: $f_6 = |T(v)|$.
- (F7) The clustering coefficient of v : $f_7 = 2f_6/(f_1(f_1 - 1))$.

Then each kind of above features are normalized to range $(0, 1)$. We construct the primary structural feature matrix $\tilde{F}$ in which each row is a 7-dimensional vector composed of normalized features. We calculate Finally, the features are aggregated by computing the mean and sum of node features in each egonet as follows:

$$F = \tilde{F} \circ \hat{D}^{-1} \hat{A} \tilde{F} \circ \hat{A} \tilde{F}, \quad (1)$$

where $\circ$ is the concatenation operator. $\hat{A} = A + I$ is the adjacency matrix of graph G with added self-loops where $I \in \mathbb{R}^{n \times n}$ is the identity matrix, and $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$. Because a small F is enough as guidance information, we only aggregates the features only once. The dimension of every node’s ultimate feature vectors is only $m = 21$. Intuitively, the full process of feature extraction is simple and efficient while high-order structural information are captured.

3.3 Graph Auto-encoder

Our encoder consists of multi layers of graph convolutional networks (GCN) [14]. The original GCN use the propagation rule as follows:

$$H^{(l)} = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l-1)} \Theta^{(l)}), \quad (2)$$

where $l = 1, 2, \dots, L$. $H^{(l-1)} \in \mathbb{R}^{n \times d}$ and $\Theta^{(l)} \in \mathbb{R}^{d \times c}$ represent the activation matrix and the trainable weight matrix of the l th layer respectively. $H^{(0)}$ is $\hat{A}$ or a randomly initialized matrix. $\sigma(\cdot)$ denotes a nonlinear activation function.

As discussed in [30], the propagation rule of original GCN is essentially a variety of mean-pooling. Sometimes mean-pooling does not perform well for distinguishing local structures, which is lethal to role-oriented tasks. For greater discrimination, we apply following sum-pooling propagation rule in our graph convolutional encoder:

$$H^{(l)} = \sigma(\hat{A} H^{(l-1)} \Theta^{(l)}). \quad (3)$$

The output of the L th layer of graph convolutional network, i.e. $H^{(L)}$, is the representation matrix whose i th row $H_i^{(L)}$ is the representation of node v_i .

Then we use multi-layer perceptrons to decode the representations as follows:

$$\hat{H}^{(s)} = \sigma(\hat{H}^{(s-1)}W^{(s)} + b^{(s)}), \quad (4)$$

where $s = 1, 2, \dots, S$. $\hat{H}^{(s-1)} \in \mathbb{R}^{n \times d'}$ is the activation matrix in the s th layer, $W^{(l)} \in \mathbb{R}^{d' \times c'}$ and $b^{(s)}$ are the trainable weight matrix and the biases of the s th-layer perceptron respectively. $\hat{H}^{(0)} = H^{(L)}$. The reconstructed structural features are generated by the last layer of perceptron:

$$\hat{F} = \hat{H}^{(S)} = \sigma(\hat{H}^{(S-1)}W^{(S)} + b^{(S)}), \quad (5)$$

which is used to construct loss function.

3.4 Training

In contrast to most existing methods which treat hand-craft structural features as inputs, we use extracted features as guidance of training for our algorithm. Thus, we let the reconstructed feature matrix $\hat{F}$ approach the extracted feature matrix F by minimizing the guidance loss which is defined as follows:

$$\mathcal{L}_g = \left\| \hat{F} - F \right\|_{fro}^2, \quad (6)$$

where $\|\cdot\|_{fro}$ denotes the Frobenius norm. In order to increase the robustness of our model, we introduce the following L2 regularization to the parameters of our graph convolutional auto-encoder:

$$\mathcal{L}_{reg} = \sum_{l=1}^L \left\| \Theta^{(l)} \right\|_{fro}^2 + \sum_{s=1}^S \left(\left\| W^{(s)} + b^{(s)} \right\|_2^2 \right). \quad (7)$$

The final loss function is as follows:

$$\mathcal{L} = \mathcal{L}_g + \lambda \mathcal{L}_{reg}, \quad (8)$$

where λ is the weight of $\mathcal{L}_{reg}$. As we discussed above, the embeddings obtained via most existing methods fit the mutually extracted features. In contrast, our approach GAS can provide embeddings containing richer structural information which can reconstruct the extracted features while minimizing the guidance loss.

3.5 Computational Complexity

Let $|V|$ denotes the number of nodes, $|E|$ denotes the number of edges, f be the number of extracted primary local and egonet features. $f = 7$ in our method. For a input real-world network, computing primary local and egonet features takes $O(|V|)$ [9]. Though we express the aggregation process as matrix multiplications,

Table 2. Detailed statistic of the datasets.

Dataset	# Nodes	# Edges	# Classes	Density(%)
Brazil	131	1,003	4	11.7792
Europe	399	5,995	4	7.5503
USA	1,190	13,599	4	1.9222
Reality-call	6,809	7,680	3	0.0331
Actor	7,779	26,752	4	0.0884

each aggregation actually take $O(|E|f)$ due to the sparsity of the network. While aggregating only once, the total time complexity of feature extraction is $O(|E|f + |V|f)$. Thus, the process of feature extraction in our framework is highly efficient.

Each layer of graph convolutional networks has complexity $O(|E|dc)$ [14], where d is the dimension of input representations and c is the dimension of output representations. Compared with other graph neural networks propagating on all the edges, i.e. without sampling in each node's neighborhood, our encoder composed of GCNs is one of the most efficient models.

4 Experiments

4.1 Datasets

To verify the effectiveness of our proposed approach GAS, we conduct experiments on following real-world networks:

- **Air-traffic networks** [20]: There are three air-traffic networks, including American, Brazilian and European air-traffic networks (shortly denoted as USA, Brazil, and Europe). In these networks, nodes represent airports and edges represent commercial flights existing between airports. Airports are grouped based on the volume of airplanes or people.
- **Actor co-occurrence network** [15]: It is an actor only network extracted from a film-director-actor-writer network [26]. In this network, nodes mean actors and edges between them mean that they have appeared on the same Wiki pages. Actors are labeled based on the number of words of their Wiki pages. For convenience, we use Actor to denote this network.
- **Phone call Network** [22]: This real phone call data records a total of 52050 calls from 6809 users during the period from September 2004 to January 2005. We express it as a static network in which nodes denote users and edges denote communication between them. The users are divided into different classes based on their frequencies of calls. For convenience, we use Reality to denote this network.

The details of these datasets are provided in Table 2.

Table 3. The micro-averaged F1 score on different datasets (The mean results of 10 runs; OM means out of memory).

Dataset	Brazil	Europe	USA	Reality	Actor
DeepWalk	0.4050 (8)	0.4064 (8)	0.4936 (8)	0.5098 (7)	0.3560 (7)
Node2vec	0.4145 (7)	0.4441 (7)	0.4988 (7)	0.4917 (8)	0.3609 (6)
RolX	0.7107 (6)	0.5495 (4)	0.6424 (2)	0.6538 (5)	0.4723 (3)
GraphWave	0.7300 (4)	0.5331 (6)	0.5180 (6)	0.8255 (3)	OM
Struc2vec	0.7262 (5)	0.5788 (2)	0.6113 (5)	0.6074 (6)	0.4679 (4)
DRNE	0.7555 (2)	0.5574 (3)	0.6159 (4)	0.8389 (2)	0.4751 (2)
Features	0.7365 (3)	0.5373 (5)	0.6227 (3)	0.7740 (4)	0.4653 (5)
GAS	0.7995 (1)	0.5825 (1)	0.6737 (1)	0.8458 (1)	0.4822 (1)

Table 4. The macro-averaged F1 score on different datasets (The mean results of 10 runs; OM means out of memory).

Dataset	Brazil	Europe	USA	Reality	Actor
DeepWalk	0.4009 (8)	0.4009 (8)	0.4763 (7)	0.3332 (7)	0.3456 (7)
Node2vec	0.3994 (7)	0.4218 (7)	0.4818 (6)	0.3240 (8)	0.3473 (6)
RolX	0.7053 (6)	0.5364 (5)	0.6352 (2)	0.4290 (5)	0.4586 (4)
GraphWave	0.7144 (5)	0.4953 (6)	0.4730 (8)	0.5180 (3)	OM
Struc2vec	0.7205 (4)	0.5752 (2)	0.6036 (4)	0.3874 (6)	0.4611 (3)
DRNE	0.7446 (2)	0.5475 (3)	0.6013 (5)	0.5244 (2)	0.4682 (2)
Features	0.7206 (3)	0.5422 (4)	0.6141 (3)	0.4986 (4)	0.4542 (5)
GAS	0.7940 (1)	0.5799 (1)	0.6658 (1)	0.5327 (1)	0.4815 (1)

4.2 Model Configuration

We adopt two layers of graph convolutional networks as the encoder and two layers of multi-layer perceptrons. For the nonlinear activation function $\sigma(\cdot)$, sigmoid function is selected. We set the dimension of embeddings to 128 and the weight of L2 regularization to 0.8. The models is initialized using Glorot initialization [4] and trained using the Adam SGD optimizer [13] with a learning rate of 0.001 for at most 200 epochs. We also use an early stopping strategy on the loss $\mathcal{L}$ with a patience of 15 epochs.

4.3 Baselines

We compared our model with several state-of-the-art network representation learning approaches including both classic models and role-oriented models. The dimension of embeddings is set to 128 to all the baselines except Features and DRNE [27]. The baselines and their parameter settings are listed as follows:

- **Features**: Features denotes the handcraft structural features referenced ReFeX [9] which we use as guidance information.
- **DeepWalk** [19]: DeepWalk learns node representations by leveraging Skip-Gram [17] to node sequences generated by random walks. We set window size to 10, walk length to 80 and each node’s number of walks to 20.
- **Node2vec** [5]: Node2vec uses a biased random walk method to capture more neighborhood information than DeepWalk. To capture more structural information, we set hyper-parameter $p = 1$ and $q = 3$. The parameters of random walks are the same as those of DeepWalk.
- **RolX** [8]: RolX decompose the feature matrix generated by ReFeX [9] using non-negative matrix factorization. For different networks, we tune the number of aggregation recursions to get the best results.
- **GraphWave** [17]: GraphWave treats spectral graph wavelets as probability distributions and generate embeddings by characterizing the distributions. We use its default settings.
- **Struc2vec** [20]: Struc2vec applies random walk method on a reconstructed multi-layer networks. We tune the layer number of the reconstructed hierarchy for different networks and the parameters of random walks are the same as those of DeepWalk.
- **DRNE** [27]: DRNE uses layer normalized LSTMs to represent nodes and captures the regular equivalence by recursively aggregating the representations. The dimension of embeddings is set as 32, with which DRNE gets the best results. The other parameters are set as default.

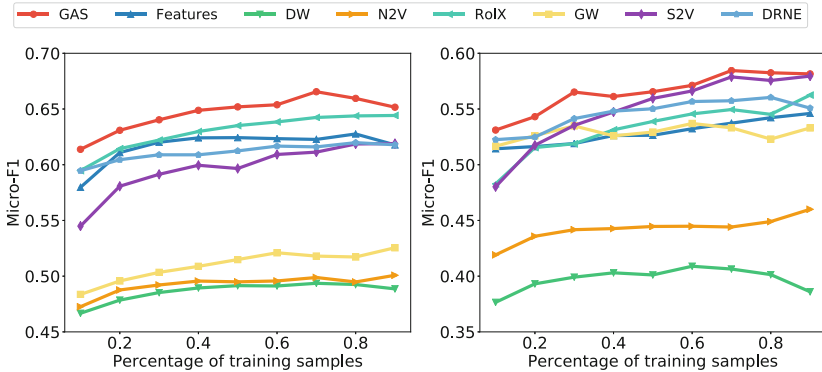


Fig. 3. The micro-averaged F1 score for role-oriented node classification on different dataset splits. Left: USA. Right: European. The shorthands DW, N2V, GW and S2V denote methods DeepWalk, Node2vec, GraphWave and Struc2vec respectively.

4.4 Role-Oriented Node Classification

We conduct role-oriented node classification tasks to evaluate the proposed graph auto-encoder guided by structural information. For each dataset, a linear logistic

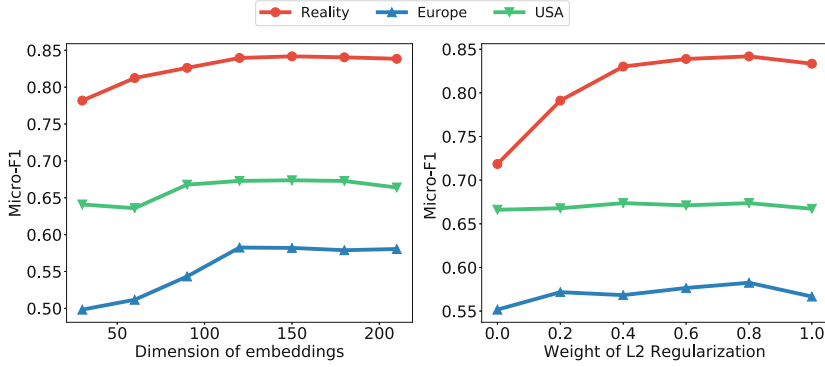


Fig. 4. Parameter Sensitivity w.r.t. the dimension of embeddings (left), the weight of L2 regularization (right).

regression classifier trained and tested using embeddings generated by each baseline and our model. We randomly sample 70% node embeddings as the training set and the other embeddings are used as the test set. The micro-averaged F1 score and macro-averaged F1 score are reported in Table 3 and Table 4 respectively, where the ranks of results are in brackets.

We have following observations:

- Role-oriented embedding methods including RolX, GraphWave and DRNE performs much better than classic embedding methods including DeepWalk and Node2vec, which verifies the no-free-lunch theorem [29] and the necessity of role-oriented network representation learning.
- Role-oriented baseline methods get fluctuating results because node roles in different networks are determined by different features. That is one of our motivation to use manually extracted features as guidance information.
- The results of our extracted features are not bad though their dimension is only 21. That indicates the extracted features contain a certain amount of structural information, which is their foundation to be used as guidance.
- Compared with Features, our approach GAS has improved performance greatly, which verifies the correctness of the idea using extracted structural features as guidance information.
- Our GAS overperforms all of the baselines on all the datasets. GAS is a state-of-the-art method for role-oriented network representation learning.

We also report the results of role-oriented node classification tasks using different train/test splits of the networks in Fig. 3. It can be observed that our model GAS still performs better than baselines on different splits.

4.5 Parameter Sensitivity

To evaluate how parameters influence the performance of our proposed GAS, role-oriented node classification tasks are conducted to datasets Reality, Europe

and USA. We illustrate the effect of embedding dimension and the weight of L2 regularization in Fig. 4.

The effect of embedding dimension is demonstrated in the left part of Fig. 4. The performance becomes better when embedding dimension increases from a small value due to more information contained. Our model achieves the best results when the dimension is 120 and shows stable performance on larger dimensions. The effect of the weight of L2 regularization is demonstrated in the right part of Fig. 4. As we can observe, the performance of GAS varies on different datasets. While the weight increasing, micro-averaged F1 score on Reality raises rapidly, grows slowly on Europe, and keeps stable on USA. It indicates these networks have different sensitivities to the quality of embeddings. All the results achieve the best when the weight is about 0.8. The score starts decreasing when the weight exceeds 0.8.

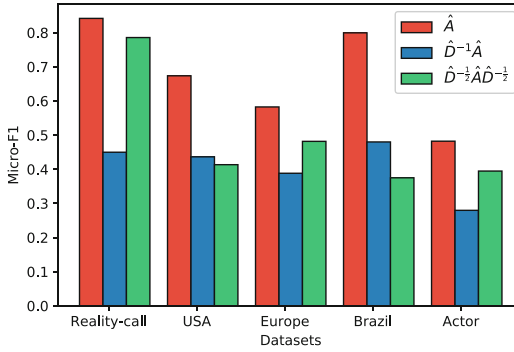


Fig. 5. Effectiveness w.r.t the propagation rules of graph convolutional encoder.

4.6 Propagation Rule Analysis

Graph neural networks could capture structural information due to its propagation mechanisms on edges. For GAS, we leverage graph convolutional networks as the encoder. And we use sum-pooling propagation rule defined as Eq. 3 for its great power of distinguishing local structures. To verify the point, we change the propagation rule of GAS to that of original GCN and mean-pooling propagation rule [6] defined as Eq. 2 and Eq. 9 respectively.

$$H^{(l)} = \sigma(\hat{D}^{-1}\hat{A}H^{(l-1)}\Theta^{(l)}). \quad (9)$$

The results are reported in Fig. 5 where we denote the three propagation rules shortly as $\hat{A}$, $\hat{D}^{-1}\hat{A}$ and $\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$. The sum-pooling performs much better than the other two methods. Original GCN and mean-pooling propagation rules perform differently due to the distinctions of datasets.

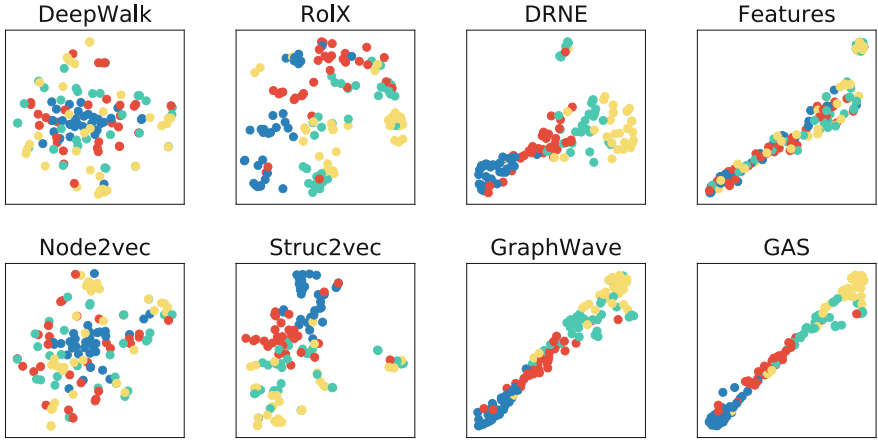


Fig. 6. Visualization of node representations on Brazil network.

4.7 Visualization

We illustrate the embeddings which are generated by our approach GAS and baselines in Fig. 6. Here, the embeddings are projected to two dimensions using t-SNE [16] and the colors of nodes represents their labels. The closer the embeddings of nodes with the same label are, the better effectiveness are demonstrated. Thus, the same observations described in Sect. 4.4 can be obtained again from Fig. 6.

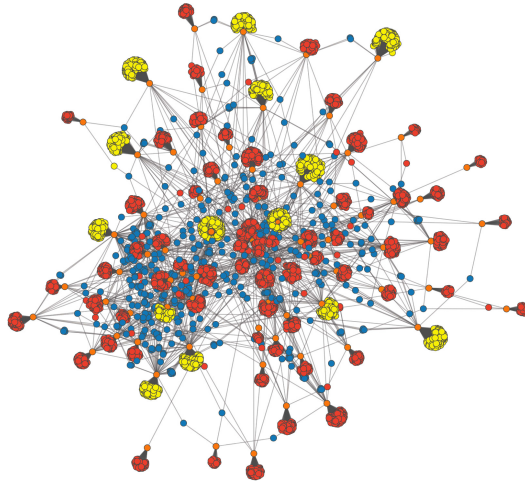


Fig. 7. The result of clustering nodes by using K-means based on embeddings.

We also adopt K-means algorithm on embeddings generated by GAS to cluster the nodes. As shown in the Fig. 7, we divide the nodes of Reality network

into four clusters which are denoted by colors. We can see that nodes in the same clusters have similar local structural patterns. For example, almost all the orange nodes are star-center nodes; blue nodes are mainly bridge nodes; though nodes in red and yellow are mainly star-edge nodes, the yellow nodes are connected to higher-degree nodes.

The above visualization experiments indicate that GAS can capture high-quality structural properties and encode them into low-dimensional embeddings.

5 Conclusion

In this paper, we discuss the importance of role-oriented network representation learning. To avoid the limitations of existing methods, we introduce a novel idea using manual extracted structural features as guidance information to train a role-oriented graph neural network model. And we implement the idea via our proposed approach GAS which leverages sum-pooling propagating graph convolutional layers as encoder and handcraft structural features as the guidance of training. The extensive experiments verify the correctness of our idea and the effectiveness of GAS.

Acknowledgments. This work was supported by the National Natural Science Foundation of China (61902278, 91746205, 91746107, 51438009), the National Key R&D Program of China (2018YFC0831000, 2018YFC0809800, 2016QY15Z2502-02) and the Science and Technology Key R&D Program of Tianjin (18YFZCSF01370).

References

1. Ahmed, N.K., et al.: role2vec: role-based network embeddings. In: DLG KDD (2019)
2. Donnat, C., Zitnik, M., Hallac, D., Leskovec, J.: Learning structural node embeddings via diffusion wavelets. In: SIGKDD, pp. 1320–1329 (2018)
3. Gilpin, S., Eliassi-Rad, T., Davidson, I.: Guided learning for role discovery (GLRD): framework, algorithms, and applications. In: SIGKDD, pp. 113–121 (2013)
4. Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: AISTATS, pp. 249–256 (2010)
5. Grover, A., Leskovec, J.: node2vec: scalable feature learning for networks. In: SIGKDD, pp. 855–864 (2016)
6. Hamilton, W.L., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. In: NIPS, pp. 1024–1034 (2017)
7. Heimann, M., Shen, H., Safavi, T., Koutra, D.: Regal: representation learning-based graph alignment. In: CIKM, pp. 117–126 (2018)
8. Henderson, K., et al.: RoLX: structural role extraction & mining in large graphs. In: SIGKDD, pp. 1231–1239 (2012)
9. Henderson, K., et al.: It’s who you know: graph mining using recursive structural features. In: SIGKDD, pp. 663–671 (2011)
10. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)

11. Holland, P.W., Leinhardt, S.: Local structure in social networks. *Sociol. Methodol.* **7**, 1–45 (1976)
12. Ke, H., et al.: Deep mutual encode model for network embedding from structural identity. *IEEE Access* **7**, 177484–177496 (2019)
13. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. In: *ICLR* (2014)
14. Kipf, T., Welling, M.: Semi-supervised classification with graph convolutional networks. In: *ICLR* (2017)
15. Ma, X., Qin, G., Qiu, Z., Zheng, M., Wang, Z.: RiWalk: fast structural node embedding via role identification. *arXiv preprint [arXiv:1910.06541](https://arxiv.org/abs/1910.06541)* (2019)
16. van der Maaten, L., Hinton, G.: Visualizing data using t-SNE. *J. Mach. Learn. Res.* **9**(Nov), 2579–2605 (2008)
17. Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J.: Distributed representations of words and phrases and their compositionality. In: *NIPS*, pp. 3111–3119 (2013)
18. Page, L., Brin, S., Motwani, R., Winograd, T.: The pagerank citation ranking: bringing order to the web. Technical report, Stanford InfoLab (1999)
19. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: online learning of social representations. In: *SIGKDD*, pp. 701–710 (2014)
20. Ribeiro, L.F., Saverese, P.H., Figueiredo, D.R.: struc2vec: learning node representations from structural identity. In: *SIGKDD*, pp. 385–394 (2017)
21. Rossi, R.A., Ahmed, N.K.: Role discovery in networks. *IEEE Trans. Knowl. Data Eng.* **27**(4), 1112–1131 (2014)
22. Rossi, R.A., Ahmed, N.K.: The network data repository with interactive graph analytics and visualization. In: *AAAI*, pp. 4292–4293 (2015)
23. Rossi, R.A., Ahmed, N.K., Koh, E.: Higher-order network representation learning. In: *WWW* (2018)
24. Rossi, R.A., Jin, D., Kim, S., Ahmed, N.K., Koutra, D., Lee, J.B.: From community to role-based graph embeddings. *arXiv preprint [arXiv:1908.08572](https://arxiv.org/abs/1908.08572)* (2019)
25. Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., Mei, Q.: Line: large-scale information network embedding. In: *WWW*, pp. 1067–1077 (2015)
26. Tang, J., Sun, J., Wang, C., Yang, Z.: Social influence analysis in large-scale networks. In: *SIGKDD*, pp. 807–816 (2009)
27. Tu, K., Cui, P., Wang, X., Yu, P.S., Zhu, W.: Deep recursive network embedding with regular equivalence. In: *SIGKDD*, pp. 2357–2366 (2018)
28. Wasserman, S., Faust, K.: *Social Network Analysis: Methods and Applications*, vol. 8. Cambridge University Press, Cambridge (1994)
29. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1**(1), 67–82 (1997)
30. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? In: *ICLR* (2019)
31. Zitnik, M., Leskovec, J.: Predicting multicellular function through multi-layer tissue networks. *Bioinformatics* **33**(14), i190–i198 (2017)